

SKILL & SKILL STACK ARCHITECTURE

Deep Audit

Multi-Perspective Analysis: Contrarian Review, Staff Engineer Code Review, YC Partner Assessment, Bottleneck Profiling, Decision Trees, and 3 Wildly Different Architectures

June 2026 | Version 1.0 | Confidential

1. Executive Summary

Is the skill and skill stack system working?

Working is doing a lot of heavy lifting in that question. The system renders 64 skills, provides search/filter/graph views, and copies install commands to clipboard. That works. But the deeper question is whether the architecture serves its stated purpose as a "Skill Stack Architecture" or whether it is, in practice, a skill catalog with delusions of orchestration. The honest answer: it is a catalog. The "stack" part is a fiction. The "pipeline" is a clipboard copy. There is no composition, no execution, no feedback loop, and no persistence beyond localStorage. The 8 MCP stacks in stacks.json are completely disconnected from the UI. Skills are hardcoded in a TypeScript file that requires a code change and redeploy to update. The entire "pipeline" is a bash script generation feature that copies text to the clipboard. If the user pastes it into a terminal and runs it, there is zero feedback, zero status tracking, and zero error handling for what happens after the clipboard.

The #1 Bottleneck (Profile Before You Optimize)

The bottleneck is not performance. The bottleneck is architectural: there is no skill execution runtime. The system has no way to know whether a skill was installed successfully, whether it is being used, or whether it conflicts with another skill. The "pipeline" is purely a text-generation feature. This means the system cannot provide the one thing that would make it genuinely useful: confidence that a skill stack works as a coherent unit. The SP-7 scoring algorithm, the heatmap, the radar charts, the design options, the compatibility matrix in stacks.json — all of these are sophisticated analysis tools that produce recommendations which the system itself cannot validate or enforce. The gap between "recommending a stack" and "verifying a stack works" is the entire value gap.

The 80/20 Version

Component	Value Delivered	Complexity Cost	Keep?
Skill catalog + search	Core value: discover and copy install commands	Low	YES
Tier architecture	Dependency ordering prevents broken installs	Low	YES
Basket + clipboard	Batch install workflow	Low	YES
Skill detail drawer	SKILL.md content, related skills	Medium	YES

Graph view	Visual, but tag-overlap connections are weak signal	High	MAYBE
SP-7 scoring + radar	Looks sophisticated, but vectors are derived, not measured	High	NO
Design options (5)	Opinionated presets, but no one can act on them	High	NO
Heatmap (16 skills)	Only covers S01-S16, not S17-S64	Medium	NO
MCP stacks (stacks.json)	Zero UI integration. Dead data.	Medium	NO (as-is)
AI Portal Gateway	Keyword routing table. Not AI.	Medium	NO

Table 1: 80/20 Analysis — What delivers value vs. what costs complexity

The 80/20 version is: a searchable skill catalog with tier ordering, a basket for batch install, and a detail drawer that shows SKILL.md content. Everything else — the SP-7 engine, the design options, the heatmap, the AI portal, the graph view — is decorative complexity that creates the appearance of sophistication without the substance. Remove those five sections and the system loses zero core functionality while shedding approximately 60% of its code complexity.

2. System Architecture Map

Live Data Paths (Active)

Source	Consumer	Transport	Status
skill-data.ts (64 skills)	SkillMarketplace, SkillReference, TierArchitecture, DesignAlgorithm, AIPortalGateway, SectionMapping, ImplementationBlueprint	Static import	ACTIVE
skill-store.ts (Zustand)	SkillCard, SkillDetailDrawer, ClipboardPanel, BasketPanel, SkillMarketplace	React context	ACTIVE
skills-local/*/SKILL.md	SkillMarkdownRenderer	API fetch + filesystem	ACTIVE

Table 2: Active data paths in the system

Dead Data Paths (No UI Consumer)

Source	Records	Consumer	Status
skills-local/stacks.json	8 MCP stacks	NONE	DEAD
skills-local/mcp-registry.json	78 MCP servers	NONE	DEAD
skills.json (root)	50+ skill manifests	NONE	DEAD
prisma/schema.prisma	User/Post models	db.ts stub (unused)	DEAD
upload/skills_trending.json	Scraped HTML	NONE	DEAD

Table 3: Dead data paths — data exists but no component consumes it

The architecture has a striking imbalance: 4 active data paths serving 12+ components, but 5 dead data paths containing the most potentially valuable data (MCP stacks, MCP registry, full skill manifests, database schema). The stacks.json file contains 8 carefully curated MCP server combinations with compatibility matrices, synergy rules, and conflict detection — none of which is surfaced in the UI. The mcp-registry.json has 78 MCP servers with ratings and download counts — again, invisible to users. The Prisma schema is a stub that logs a warning when

initialized. These represent the largest untapped value in the codebase.

The Two-Registry Problem

There are two completely independent skill registries that are never synchronized: `skill-data.ts` (64 curated skills with tier/category/tag metadata) and `skills.json` at the project root (50+ skills in the `agentskills.io` format with `directory/type/has_scripts` metadata). These use different schemas, different ID systems, and different categorization schemes. Any change to one must be manually replicated to the other, and there is no validation that they stay in sync. This is a classic dual-write problem that will silently diverge over time.

3. The Contrarian View

Find the Flaw First

The fundamental flaw: this system confuses curation with orchestration. It treats skills as catalog items to be browsed and collected, when their actual value comes from composition and execution. A skill is not a product you add to a shopping cart. A skill is a behavioral contract — it specifies how an AI agent should think and act in a particular domain. The current architecture has no model for what happens when skills interact, conflict, or compose. The tier system (T0-T3) implies dependency ordering, but the system never enforces or even checks these dependencies at install time. The "pipeline" generates a bash script with "set -e" and hopes for the best.

What a Contrarian Would Say

Contrarian Claim	Confidence	Second-Order Effect
"This is a brochure, not a product. Nobody needs a brochure for CLI commands."	9/10	Users will copy the install command once and never return — zero retention
"The SP-7 scoring is numerology. The vectors are derived from tier+category, not measured from anything real."	9/10	If someone relies on SP-7 for decisions, they are optimizing a phantom
"The 'pipeline' is an insult to the word. It generates a bash script. That is not a pipeline."	8/10	Users who expect pipeline semantics (stages, status, rollback) will be confused and disappointed
"64 skills is not a marketplace. It is a curated list. Calling it a marketplace implies supply/demand dynamics that do not exist."	7/10	The label creates expectations of community contribution, ratings, reviews — none of which exist
"The MCP stacks in stacks.json are the most valuable data in the repo, and they are completely invisible. This is like hiding your best product."	9/10	The stacks could be the killer feature, but they are invisible and decaying
"Hardcoding 64 skills in a TypeScript file is a deployment bottleneck. Every skill update requires a full rebuild and redeploy."	8/10	Skill authors cannot self-publish; the system cannot scale beyond manual curation

Table 4: Contrarian claims with confidence levels and second-order effects

Steelmanning the Opposite View

The strongest argument against the contrarian: this is a specification and reference architecture, not a production SaaS. It is designed to be a living, navigable specification that demonstrates how skills should be organized, scored, and selected. The 13-section page is a design document that happens to be executable. The SP-7 vectors are not meant to be empirically measured — they are design intent encoded as numbers, a common pattern in architecture decision records. The hardcoded skills ensure consistency and quality control that a dynamic registry would not. The "pipeline" is intentionally minimal because the actual execution happens in the AI agent's environment, not in this web app. The brochure IS the product — it is a specification that other implementations follow.

Counter to the steelman: Even as a specification, the system should demonstrate the behaviors it prescribes. A specification for skill stack orchestration that cannot orchestrate skill stacks is a specification with unimplemented requirements. The gap between specification and demonstration undermines credibility. If you are going to prescribe SP-7 scoring, the scores should be reproducible and empirically grounded, not decorative. If you are going to define MCP stacks with compatibility matrices, the UI should render those stacks and validate them. A specification that ignores its own recommendations is not authoritative — it is aspirational.

4. Profile Before You Optimize — The Actual Bottleneck

EXPLAIN ANALYZE Output Interpretation

If we ran EXPLAIN ANALYZE on the skill system, the query plan would look like this: the most expensive operation is not the SKILL.md fetch (one API call, filesystem read, cached by browser for the session), not the Zustand store updates (in-memory, synchronous), and not the search/filter (client-side array operations on 64 items). The most expensive operation is the gap between recommendation and action. Every piece of sophisticated analysis (SP-7 scores, design options, compatibility matrices) produces a recommendation that the user must manually interpret, manually execute, and manually verify. The cost is not computational — it is cognitive. The user does the work that the system should do.

Operation	Cost	Row s	Bottleneck Type	Actual Time
Skill catalog render	0.01	64	Static import	<1ms
Search/filter	0.02	0-64	Client array scan	<1ms
SKILL.md fetch	1.00	1	Network + filesystem	50-200ms
Zustand store update	0.01	1	In-memory	<1ms
SP-7 vector compute	0.05	7	Tier+category lookup	<1ms
Pipeline script gen	0.01	1	String template	<1ms
User interprets recommendation and executes manually	INFINITE	?	Cognitive gap	Minutes-hours

Table 5: EXPLAIN ANALYZE — the real bottleneck is cognitive, not computational

Metrics a Data Engineer Would Add

The current system has zero observability. No events are tracked. No metrics are collected. No dashboards exist. A data engineer would look at this and immediately identify that you cannot improve what you do not measure. The following metrics are ranked by impact — implementing the top 3 would provide 80% of the observability value with 20% of the instrumentation effort.

Metric	Event Type	Impact Rank	Schema Field
--------	------------	-------------	--------------

skill_install_initiated	Button click	1	{skillId, source, timestamp}
skill_detail_opened	Drawer open	2	{skillId, tab, timestamp}
basket_export_format	Export click	3	{format, skillCount, skillIds[], timestamp}
search_query	Input change	4	{query, resultCount, filters, timestamp}
skill_md_fetch_duration	API response	5	{skillDir, duration, status, contentLength}
graph_node_clicked	SVG click	6	{skillId, connectionCount, timestamp}
pipeline_run_clicked	Button click	7	{skillCount, skillIds[], timestamp}
category_filter_applied	Tab click	8	{category, resultCount, timestamp}
clipboard_copy_success	API result	9	{method (clipboard/fallback), skillId, timestamp}
design_option_selected	Card click	10	{optionId, timestamp}

Table 6: Data engineer metrics schema, ranked by impact

5. Three Wildly Different Approaches

Approach A: The Registry (Current, Refined)

Keep the catalog model but make it authoritative. Move skills from hardcoded TypeScript to a database (Prisma + SQLite). Add a CRUD API. Surface the MCP stacks from stacks.json in the UI. Add install tracking (did the npx command succeed?). Add user ratings and usage counts. The "pipeline" becomes an install orchestrator that runs commands and reports status. This is the path of least resistance — it preserves 90% of the existing code while filling the largest gaps. The 80/20 version of this approach: add Prisma models for skills/stacks, migrate the 64 skills to seed data, create 3 API endpoints (list, detail, install-status), and render the 8 MCP stacks as a new section.

Approach B: The Runtime (Composition Engine)

Abandon the catalog metaphor entirely. Instead of listing skills, the system becomes a composition engine that takes a user goal as input and produces a validated skill stack as output. The user describes what they want to build ("I need a content marketing pipeline with image generation and SEO"), and the system uses the compatibility matrix from stacks.json, the tag overlap data, and the tier dependencies to generate a recommended stack, validate it against known conflicts, and output a single install command. This is the "paved path" approach — opinionated, validated, and safe. The user never browses 64 skills because the system does the composition for them. The 80/20 version: a single input field that maps user intent to one of the 8 pre-built MCP stacks using the intentRoutes table, with conflict validation and a one-click install button.

Approach C: The Protocol (Decentralized Registry)

Abandon the monolith entirely. Publish skills as individual npm packages following a standard skill schema (the agentskills.io format already defined in skills.json). The web app becomes a thin registry client that fetches skill metadata from npm, renders it, and delegates installation to the user's package manager. MCP stacks are published as meta-packages (e.g., @skill-stacks/creative-studio) that declare skill dependencies. This is the most scalable approach — it allows community contribution, version management, and dependency resolution via existing npm tooling. The 80/20 version: publish the 64 skills as npm packages with the existing SKILL.md as the README, and replace the hardcoded skill-data.ts with a dynamic fetch from the npm registry API.

Dimension	A: Registry (Refined)	B: Runtime (Composition)	C: Protocol (Decentralized)
-----------	--------------------------	-----------------------------	--------------------------------

Core metaphor	Enhanced catalog	Goal-to-stack engine	npm-like registry
Scalability	Limited (manual curation)	Medium (fixed stack presets)	Unlimited (community)
Implementation effort	Low (2-3 weeks)	Medium (4-6 weeks)	High (8-12 weeks)
User complexity	Browse + select (familiar)	Describe goal (low friction)	npm install (developer-familiar)
Validation capability	Post-install status only	Pre-install conflict check	npm dependency resolution
Moat	Low (anyone can curate)	Medium (compatibility data)	High (network effects)
Confidence this works	9/10	7/10	5/10
Steelmanned opposite	Does not solve the composition problem; still a catalog	Hard to cover edge cases; AI intent parsing is unreliable	Community may not materialize; npm is not designed for skills
Second-order effect	Maintenance burden grows with skill count	Users trust the system and stop thinking critically about stack choices	Skills become packages; versioning hell, breaking changes propagate

Table 7: Three wildly different approaches — comparison matrix

6. Decision Tree: If X Then Y

The following decision tree maps every branch for skill and skill stack operations. Each branch represents a distinct execution path through the system, traced from user interaction through state changes to final output.

If	Then	State Change	External Dep
User clicks skill card	Open detail drawer	selectedSkillId = skill.id	None
User clicks SKILL.md tab	Fetch markdown from API	fetchReducer: FETCH_START then FETCH_SUCCESS	/api/skill-content
API returns 404 for SKILL.md	Show "No SKILL.md available" with retry button	fetchReducer: FETCH_ERROR	Filesystem
User clicks Copy on skill card	Copy installCommand to clipboard	addToClipboard() + setCopied(true)	navigator.clipboard
Clipboard API unavailable (HTTP)	Fallback: textarea + execCommand	Same state change	document.execCommand
User clicks Add to Basket	Add skillId to basket array	addToBasket(skillId)	None
Skill already in basket	No-op (dedup)	No state change	None
User clicks Run Pipeline	Generate bash script from basket	setCopied(true)	navigator.clipboard
Basket is empty	No-op (button disabled)	No state change	None
User clicks Export in Basket	Format as script/commands/JSON	setCopiedType(format)	navigator.clipboard
User searches by keyword	Filter skills array by name/role/tags/category	search state update triggers useMemo recompute	None
User switches to Graph view	Render SVG force-directed graph	viewMode = "graph"	None
User switches to Pipeline view	Render tier-grouped flow	viewMode = "pipeline"	None
User selects category tab	Filter by category	activeCategory update	None

User clicks MCP stack (CURRENTLY IMPOSSIBLE)	No UI exists for stacks.json	No state change	N/A
--	---------------------------------	-----------------	-----

Table 8: Decision tree — all branches mapped

The decision tree reveals that 15 of the 16 branches are fully client-side with no external dependencies beyond the clipboard API and the single `/api/skill-content` endpoint. This means the system has extremely low coupling to external services, which is a strength for reliability but a weakness for value creation — there is no feedback loop from the real world. The system cannot know if a skill install succeeded, if a stack works together, or if a user found the recommendation useful. The 16th branch (MCP stack interaction) is currently impossible because no UI component consumes `stacks.json`.

7. End-to-End Simulation Trace

Trace one complete execution path: a user wants to install a GSAP Animations skill as part of a creative workflow.

Step	Action	Component	State/API
1	User types "gsap" in search field	SkillMarketplace	setSearch("gsap") triggers useMemo
2	useMemo filters skills array: name.includes("gsap") or tags.includes("gsap")	SkillMarketplace	filtered = [S05, S63] (2 results)
3	AnimatePresence renders 2 SkillCard components with motion.div wrappers	SkillCard (x2)	Each card: skill prop + useSkillStore
4	User clicks S05 "GSAP Animations" card	SkillCard	setSelectedSkill("S05")
5	Sheet drawer opens, selectedSkill resolved via skills.find()	SkillDetailDrawer	skill = skills[4] (S05)
6	User clicks SKILL.md tab	SkillDetailDrawer	Tab switch renders SkillMarkdownRenderer
7	skillDir derived from installCommand: "gsap-animations"	SkillDetailDrawer	skillDir passed as prop
8	useEffect triggers fetch: GET /api/skill-content?skill=gsap-animations	SkillMarkdownRenderer	dispatch(FETCH_START), AbortController created
9	API sanitizes skill name (regex check), reads skills-local/gsap-animations/SKILL.md	/api/skill-content	readFile() returns UTF-8 string
10	Response: {skill, content, length} dispatched to reducer	SkillMarkdownRenderer	dispatch(FETCH_SUCCESS, {content, length})
11	ReactMarkdown renders content with remark-gfm + prose styling	SkillMarkdownRenderer	DOM update: SKILL.md content visible
12	User clicks "Copy Command" toolbar button	SkillMarkdownRenderer	handleCopyCommand() called
13	try: navigator.clipboard.writeText(installCommand)	SkillMarkdownRenderer	Clipboard API call

14	User clicks "Add to Basket" button in Overview tab	SkillDetailDrawer	addToBasket("S05")
15	Zustand persist writes basket to localStorage key "skill-marketplace-store"	skill-store.ts	localStorage.setItem()
16	User navigates to Pipeline view, clicks "Run Pipeline"	PipelineView	handleRunPipeline() generates bash script
17	Script copied to clipboard. User pastes into terminal. System has zero visibility into what happens next.	N/A	SYSTEM BOUNDARY — NO FEEDBACK

Table 9: End-to-end simulation trace — GSAP skill install path

The trace reveals the critical boundary at step 17: the system loses all visibility once the user copies the script to their clipboard. There is no callback, no webhook, no status check, no follow-up. The system has invested significant complexity in recommending and composing skill stacks, but has zero ability to verify whether those recommendations were correct. This is the architectural equivalent of a GPS that gives directions but never learns whether you reached your destination.

8. Staff Engineer Code Review

Design & Visual Critique

Element	Current State	Verdict	Confidence
Skill card icons (emoji)	11 categories x 1 emoji each. Emoji rendering inconsistent across browsers/OS.	Replace with SVG icons or category color dots. Emoji is amateur-hour.	8/10
Typography	font-serif for headings (browser serif), font-sans for body. No defined type scale.	No typographic hierarchy. Pick 2 fonts, define a modular scale, enforce consistently.	9/10
Color system	11 category colors. Some nearly identical (System #A8B2D8, Utility #A8B2D8). No dark mode strategy.	Reduce to 4-5 colors. Derive variants from base. The 11-color palette is visual noise.	8/10
Green pulse dot on cards	animate-ping on a 2px dot to indicate "copyable". Animated element per card x 64 cards.	Remove. It is distracting and serves no information purpose that a static indicator would not.	9/10
Graph view (SVG)	800x800 SVG. Categories in a ring. Tag-overlap connections. Zoom/pan/drag.	Most complex component with weakest signal. Tag overlap >= 2 is noisy. Consider removing.	6/10
SP-7 radar chart	Recharts RadarChart in detail drawer. 7 dimensions derived from tier+category.	Chart shows derived data, not measurements. Misleading. Remove or label as "estimated".	8/10

Table 10: Design and visual critique

Code-Level Findings

Finding	Severity	File	Fix
---------	----------	------	-----

Clipboard pattern duplicated 9x inline instead of using shared copyToClipboard()	Medium	SkillCard, SkillDetailDrawer, SkillMarkdownRenderer (x2), ClipboardPanel (x2), PipelineView, BasketPanel, SkillMarketplace	Replace all inline patterns with import from @/lib/clipboard
tierColors defined independently in SkillMarketplace and SkillCard	Low	SkillMarketplace.tsx, SkillCard.tsx	Extract to shared constant
SP-7 vectors computed at runtime from tierBaseVectors + categoryAdjustments, not stored on Skill object	Medium	SkillDetailDrawer.tsx	Pre-compute and store on Skill, or remove SP-7 entirely
Mock install counts ("500K+", "200K+") presented as real data	High	SkillDetailDrawer.tsx (tierInstallCounts)	Remove or clearly label as estimates
No caching on SKILL.md content — every drawer open is a fresh fetch	Medium	SkillMarkdownRenderer.tsx	Add SWR/React Query with stale-while-revalidate
No rate limiting on /api/skill-content	Medium	api/skill-content/route.ts	Add simple rate limit middleware (e.g., 60 req/min per IP)
heatmapData only covers S01-S16 (16 of 64 skills)	Medium	skill-data.ts	Either extend to all 64 skills or remove the heatmap section
BasketPanel resolves skills via skills.find() with silent .filter(Boolean) for missing	Low	BasketPanel.tsx	Log warning when skill ID not found in registry

Table 11: Staff engineer code review findings

9. YC Partner Review

Is this idea investable? No — not in its current form. Here is why, and what would change the answer.

Question	Assessment	Confidence
Is there a real problem?	Yes. AI agent skill discovery and composition is genuinely hard. Developers waste hours finding, evaluating, and debugging skill combinations. The MCP stack compatibility problem is real and getting worse as the ecosystem grows.	8/10
Is this the right solution?	Partially. A skill registry is necessary but not sufficient. The current implementation is a catalog, not a platform. The value is in validation and composition, not in listing.	7/10
Is there a moat?	Currently, no. Anyone can curate a list of 64 skills. The compatibility matrix in stacks.json is the beginning of a moat, but it is hidden and unvalidated. The moat would be: validated stacks with install success rates, community contributions, and integration depth with AI agent frameworks.	6/10
Can this scale?	Not with hardcoded TypeScript. The system cannot accept community contributions, cannot handle version conflicts, and cannot grow beyond manual curation. A database-backed registry with a contribution pipeline is table stakes for scale.	9/10
What is the business model?	Unclear. Free skill curation has no revenue. Premium validated stacks? Enterprise skill governance? Skill marketplace with transaction fees? The current system does not suggest any of these.	7/10
What would change the answer?	Three things: (1) Validate stacks with real install success rates, creating data that nobody else has. (2) Open skill contribution to the community, creating network effects. (3) Integrate with AI agent frameworks (OpenCode, Claude Code, Cursor) as the default skill discovery layer, creating distribution.	7/10

Table 12: YC partner assessment

What Am I Missing or Not Asking?

The questions not being asked are more revealing than the ones that are. Here are the six questions this audit has not addressed that would materially change the analysis:

1. Who is the actual user? The system seems designed for developers who use AI coding agents (OpenCode, Claude Code). But it is built as a web app, which developers visit briefly and leave. The real integration point is inside the agent, not the browser. Why is this a website instead of an MCP server that agents can query directly?

2. What happens after install? The system treats skill installation as the end of the journey. But the real value of a skill is in how it changes agent behavior. Is there any mechanism to verify that an installed skill actually affects agent behavior? If not, the entire pipeline is faith-based.

3. How do skills conflict? The stacks.json has a conflict matrix for MCP servers. But what about skill-level conflicts? Can two skills give contradictory instructions to an agent? What happens when "Chromatic Minimal" and "Neo-Industrial" are both installed? The system has no model for skill-level conflicts.

4. What is the update story? Skills are installed via npx, which fetches the latest version. But the web app shows a static snapshot. If a skill author updates their SKILL.md, the web app will not reflect it until the next code deploy. There is no webhook, no polling, no freshness indicator.

5. What is the uninstall story? The system has no concept of uninstall. If a skill causes problems, the user is on their own. A registry that cannot help you undo what it helped you do is dangerous.

6. Where is the telemetry? Not analytics for the sake of analytics, but operational telemetry. Which skills are actually installed and working? Which stacks fail? Which skills are never used? Without this feedback loop, curation is blind guessing.

10. What Can Be Deleted Without Losing Value

Component	Lines of Code (est.)	Value Lost	Confidence
DesignAlgorithm.tsx (SP-7 scoring engine)	~300	Decorative — scores are derived, not measured. No one acts on them.	9/10
Options Showcase section (5 design options)	~200	Opinionated presets with no enforcement mechanism. Pure display.	8/10
Heatmap section (16 skills only)	~150	Covers only 25% of skills. Incomplete = misleading.	9/10
AIPortalGateway.tsx (keyword routing)	~200	A static keyword matching table pretending to be AI. No actual AI.	8/10
ComparisonMatrix section	~100	Compares 5 design options that cannot be acted upon. Display only.	7/10
SkillGraph.tsx (SVG graph)	~250	Weakest signal (tag overlap ≥ 2 is noisy). Most complex component.	6/10
Mock install counts (tierInstallCounts)	~5	Fake data presented as real. Actively misleading.	10/10
Prisma schema (User/Post models)	~30	Completely unused stub. db.ts logs a warning.	10/10
skills.json at root (agentskills.io format)	~2000	Duplicate registry, different schema, no consumer. Source of confusion.	8/10
upload/skills_trending.js on	~5000	Scraped HTML payload. No consumer. No processing.	10/10

Table 13: Deletion candidates — what can be removed without losing core value

Total estimated code reduction: approximately 8,235 lines of code, configuration, and data files. The remaining system would be: skill-data.ts (64 skills), SkillMarketplace (grid view + search/filter/tier tabs), SkillCard, SkillDetailDrawer (overview + SKILL.md tabs), BasketPanel, ClipboardPanel, SkillReference table, ImplementationBlueprint, TierArchitecture, and the /api/skill-content endpoint. This is a focused, high-value system that does one thing well: help developers discover, evaluate, and install AI agent skills. Everything else is scope creep that dilutes the core value proposition.

11. Data Engineer Metrics Schema

The current Prisma schema has only User and Post models, neither of which is used. A data engineer would replace these with models that capture the actual domain: skills, stacks, installations, and user interactions. The following schema is ranked by implementation impact — the top 3 models provide 80% of the analytical value.

Model	Key Fields	Impact Rank	Enables
SkillInstall	skillId, userId, status (success/fail), duration, error, timestamp	1	Install success rates, failure patterns, skill reliability scoring
SkillView	skillId, userId, source (search/browse/detail), tab, duration, timestamp	2	Discovery funnel, skill popularity, time-on-page analytics
StackComposition	stackId, skillIds[], userId, validationStatus, conflicts[], timestamp	3	Stack usage patterns, conflict frequency, composition validation
SearchEvent	query, filters, resultCount, clickedSkillId, timestamp	4	Search quality, zero-result rate, intent classification training data
SkillFeedback	skillId, userId, rating (1-5), review, timestamp	5	Community quality signals, curation validation, ranking improvements
SkillVersion	skillId, version, changelog, breakingChanges, timestamp	6	Version management, breaking change detection, compatibility tracking
ConflictReport	skillA, skillB, conflictType, severity, description, timestamp	7	Empirical conflict data replacing theoretical compatibility matrix

Table 14: Data engineer metrics schema, ranked by impact

12. External Dependencies & Sub-Agent Breakdown

External API and Service Dependencies

Dependency	Type	Risk	Failure Mode
navigator.clipboard API	Browser API	Low	Unavailable in HTTP/iframe; fallback to textarea works
/api/skill-content (filesystem read)	Internal API	Low	Returns 404 if SKILL.md missing; graceful degradation in UI
localStorage (Zustand persist)	Browser storage	Low	Quota exceeded on 20 clipboard + N basket items; unlikely
npx skills add (external CLI)	External CLI	High	Network failure, npm registry downtime, package not found — NO FEEDBACK
Framer Motion	npm package	Low	Bundle size; animation jank on low-end devices
Recharts	npm package	Low	Used only for radar chart in detail drawer; could be replaced with CSS
react-markdown + remark-gfm	npm package	Low	XSS risk if SKILL.md contains malicious HTML; currently no sanitization

Table 15: External dependencies and failure modes

Sub-Agent Task Breakdown

If this audit were executed by autonomous sub-agents, the following decomposition would maximize parallelism while minimizing coordination overhead. Each agent has a specialized role with clear inputs and outputs.

Agent	Role	Input	Output
Profiler	Identify actual performance bottlenecks via measurement	Running app URL, Chrome DevTools	Flame chart, slowest components, render counts, bundle size analysis
Architect	Evaluate structural integrity and propose refinements	Source code, data flow map	Architecture diagram, coupling metrics, dead code report, migration plan
Security	Audit for XSS, path traversal, injection, and data exposure	API routes, component props, user inputs	Vulnerability report, severity ratings, fix recommendations

Data	Design metrics schema and validate data integrity	Current Prisma schema, runtime data	New schema, seed data, migration scripts, analytical queries
UX	Evaluate user flow, accessibility, and information architecture	Running app, component tree	Heuristic evaluation, accessibility audit (WCAG), flow optimization
Simplifier	Identify and remove unnecessary complexity	Full codebase	Deletion candidates, complexity metrics, 80/20 analysis, reduced scope proposal

Table 16: Sub-agent task breakdown

13. Consolidated Findings & Impact Rankings

#	Finding	Impact	Confidence	Priority
1	No skill execution runtime — system cannot validate its own recommendations	Critical	9/10	P0
2	MCP stacks (stacks.json) are dead data — zero UI integration	Critical	10/10	P0
3	Hardcoded skills require code deploy to update	High	9/10	P1
4	Two-registry problem (skill-data.ts vs skills.json)	High	8/10	P1
5	Mock install counts presented as real data	High	10/10	P1
6	No caching on SKILL.md fetch	Medium	8/10	P2
7	Clipboard pattern duplicated 9x instead of using shared utility	Medium	9/10	P2
8	Heatmap covers only 16/64 skills	Medium	9/10	P2
9	No rate limiting on API routes	Medium	7/10	P2
10	SP-7 scoring is derived, not measured — potentially misleading	Medium	9/10	P2
11	No skill-level conflict detection (only MCP server conflicts)	Medium	7/10	P2
12	react-markdown renders SKILL.md without XSS sanitization	Medium	6/10	P2
13	5 sections (SP-7, options, heatmap, AI portal, comparison) deliver low value for high complexity	Low	8/10	P3
14	11-category color palette with duplicates; no dark mode	Low	7/10	P3

Table 17: Consolidated findings ranked by impact with confidence levels

Priority Recommendations

P0 — Fix Immediately (Week 1): Surface the 8 MCP stacks from stacks.json as a new UI section. This is the highest-value, lowest-effort change: the data already exists, the compatibility matrix already exists, and the stacks already represent validated compositions. A simple card grid showing stack name, servers, synergy description, and install button would transform the system from a catalog into a stack recommendation engine. Simultaneously, remove the mock install counts — fake data erodes trust.

P1 — Fix Soon (Weeks 2-3): Migrate skills from hardcoded TypeScript to a Prisma database with seed data. This enables CRUD APIs, community contribution pipelines, and eliminates the deploy bottleneck. Consolidate the two registries into a single source of truth. Add install-status tracking via a simple POST endpoint that receives success/failure callbacks from the CLI.

P2 — Fix Eventually (Month 2): Add SWR caching for SKILL.md content. Consolidate the 9 inline clipboard patterns into the shared copyToClipboard utility. Extend the heatmap to cover all 64 skills or remove it. Add rate limiting to API routes. Add a "derived/estimated" label to SP-7 scores if kept. Implement basic XSS sanitization for SKILL.md content rendering.

P3 — Consider (Month 3+): Evaluate whether the 5 low-value sections (SP-7 engine, design options, heatmap, AI portal, comparison matrix) should be removed or redesigned. Reduce the color palette from 11 categories to 4-5 with derived variants. Implement dark mode. Replace emoji icons with SVG. These are polish items that should only be addressed after the P0-P2 items are resolved.

The version a senior dev would write: a searchable skill catalog backed by a database, with tier-enforced dependency ordering, a basket that generates validated install scripts, a detail drawer that shows real SKILL.md content, and an MCP stacks section that surfaces the most valuable data in the repository. Everything else is scope creep. Ship that, measure what users actually do with it, and iterate based on data — not based on what looks sophisticated in a demo.